

ML Specification Report — playground-series-s6e1

Student Exam Score Prediction • AIDE agent (20-step tree search, Claude Sonnet via shim)

Metrics, step counts, timings and every leaderboard figure are taken verbatim from the run's `facts_aide.json` (derived from `journal.json` for run `2-private-pink-mamba`). Pipeline description comes from AIDE's own auto-generated `report.md` and from `best_solution.py`; task metadata from `config.yaml`. Figures that exist only in a node's plan/analysis text are attributed inline.

Contents	
Student Exam Score Prediction • AIDE agent (20-step tree search, Claude Sonnet via shim)	1
Overview	1
Data	2
Models & Architecture	3
Training procedures	4
Search trajectory	4
Specification fields	6
Inference procedures	7
Evaluation & Benchmarking	7

Overview

The task is to predict a student's `exam_score` from study habits and demographics — a continuous regression scored locally on **R^2 (maximize)**. It is solved by **AIDE**, a tree-search coding agent that writes a complete solution script each step, executes it, reads back the printed CV score or the traceback, and mutates. Given a **20-step budget**, AIDE's champion appeared at **step 18** with **5-fold CV R^2 0.7868** — three gradient-boosting base learners (LightGBM, XGBoost, CatBoost) feeding a shallow LightGBM meta-stacker that also sees the raw numeric features.

That submission placed **1088 / 4319 — the 74.8th percentile**, with recorded leaderboard values of **Public 8.69449 / Private 8.72191**. On the private split AIDE finishes **second of three lanes**: our from-scratch agent at 8.71733, AIDE at 8.72191, NVIDIA at 8.73239. Both `local_winner` and `lb_winner` are recorded as **mine**.

Source note on the metric. `facts_aide.json`'s score table labels the metric `R2` with `direction: max`, which matches `config.yaml` and matches what AIDE actually optimized locally (`r2_score` , higher-is-better, champion 0.7868). The leaderboard columns in that same table are on a ~8.7 scale, and the recorded `lb_winner` — our agent, at 8.71733 — is the **smallest** of the three private values. The

leaderboard is therefore scored on a minimize-direction error metric, not on the R^2 AIDE maximized locally. The two scales are not comparable, and this report presents them side by side without attempting to reconcile them; all "lower is better" reasoning below applies only to the LB columns.

Why it matters. Predicting exam outcomes from study habits, attendance and sleep is the quantitative half of an early-warning system for student support — the point of a model like this is not the score itself but identifying, before the exam, which students the current pattern of behaviour is failing. It is also a dataset where the honest answer is boring: `study_hours` carries most of the signal, and the interesting question is how much a 20-step agent search can add on top of "fit a GBDT to study hours".

Data

Purpose of Data. Predict a continuous exam score — a regression problem with a clean, dense feature set. **Data Format** is **tabular CSV**, all columns either numeric or low-cardinality categorical. **Data Volume:** the competition config records **630K train rows / 270K test rows**, with a target spanning **19.6 to 100** and a mean of **62.5**. This is the largest training set in the batch, and the run's timeout pattern is a direct consequence of it.

Data Quality is essentially clean: the config records **no missing values**, and correspondingly `best_solution.py` contains no imputation logic at all — categoricals are simply cast to a shared `category` dtype with train/test category sets aligned, and numerics pass through untouched. The dominant signal is `study_hours` at $r \approx 0.76$, strong enough that the competition is largely about squeezing the residual. The champion's own node analysis records how flat the model space is: base-learner CV R^2 of **LGB 0.78594, XGB 0.78619, CatBoost 0.78492** — three different boosters within about a thousandth of each other.

Annotation Guidelines. The label is `exam_score`, continuous and bounded above by 100; predictions are real-valued and unconstrained in the submission — no clipping to the observed range is applied. R^2 and squared error agree on the optimum (R^2 is an affine rescaling of MSE for a fixed target), so the local selection metric and the training loss are consistent even though the boosters are all trained on RMSE.

Feature Set. The champion uses the **11 raw features and nothing else** — no interactions, no ratios, no target encoding of the base learners:

Group	Features
Numeric	<code>age</code> , <code>study_hours</code> , <code>class_attendance</code> , <code>sleep_hours</code>
Categorical (low cardinality)	<code>gender</code> , <code>course</code> , <code>internet_access</code> , <code>sleep_quality</code> , <code>study_method</code> , <code>facility_rating</code> , <code>exam_difficulty</code>
Encoding per learner	LightGBM: native <code>category</code> dtype with categories aligned across train/test • XGBoost: integer <code>cat.codes</code> , with unseen (-1) mapped to NaN • CatBoost: string-cast columns via <code>cat_features</code> indices

Group	Features
Meta-stacker inputs	the three base OOF predictions plus the four raw numeric columns

The absence of engineered features is a result, not an oversight. AIDE built interaction and ratio terms (`study_hours × class_attendance`, `study_hours / sleep_hours`, `age × class_attendance`, squared terms) and tested them twice. At step 16, added to the full 5-fold ensemble, they scored **0.78611** against the incumbent **0.7866** — a clear regression, and they were dropped. A second, cleaner re-test at step 14 never returned a number because it timed out. Target encoding fared slightly better but only in one position: as a base-learner input it did not help, while restricted to the meta-stacker it reached **0.78652** at step 17, still short of the incumbent.

Splitting strategy. A `KFold(n_splits=5, shuffle=True, random_state=42)` for the three base learners, and a **second, independent `KFold(n_splits=5, shuffle=True, random_state=123)`** for the meta-stacker. Using a different seed for the stacking layer is the right instinct — it decorrelates the meta-model's own validation partition from the partition that generated its inputs — though it does not fully remove the standard stacking optimism, since the OOF predictions the stacker consumes were themselves produced under the seed-42 partition it is now re-splitting. Fold count is load-bearing in this competition: three nodes (9, 10, 15) fell back to 3-fold CV purely to fit the compute budget and all landed at 0.78547–0.78556, and AIDE's report attributes the ~0.001 R^2 cost directly to the reduced training data per model. **Leaderboard status: submitted and scored**, on the different metric scale noted above.

Models & Architecture

Purpose of Architecture. A continuous exam-score regressor maximizing R^2 . **Architecture Type** is a **two-level stack**: three gradient-boosted decision-tree base learners, then a shallow LightGBM meta-learner. This is the only genuinely two-level architecture in this batch of four AIDE runs, and it is the one place where AIDE's search found something a fixed-blend pipeline would have missed — its report records the non-linear stacker beating both equal-weight and linear-regression-optimized blends of the same three models.

Input Format / Input Dimension. Level 1: the 11 raw features per row, encoded three different ways for the three learners. Level 2: a 7-column frame — `pred_lgb`, `pred_xgb`, `pred_cat` plus the four raw numerics (`age`, `study_hours`, `class_attendance`, `sleep_hours`) — so the stacker can modulate its correction by where in feature space the base models disagree, rather than applying one global weighting.

Architecture Description. The base learners are `lgb` with `objective="regression"`, `metric="rmse"`, `learning_rate=0.03`, `num_leaves=63`, `max_depth=-1`, `min_data_in_leaf=50`, `feature_fraction=0.8`, `bagging_fraction=0.8`, `seed=42`, up to 2500 rounds, early stopping 80; `xgb` with `objective="reg:squarederror"`, `eta=0.03`, `max_depth=6`, `min_child_weight=5`, `subsample=0.8`,

`colsample_bytree=0.8, tree_method="hist", seed=42`, up to 2500 rounds, early stopping 80; and `CatBoostRegressor(iterations=2000, learning_rate=0.04, depth=6, loss_function="RMSE", early_stopping_rounds=80, random_seed=42)`. The meta-learner is a deliberately tiny LightGBM — `learning_rate=0.05, num_leaves=7, min_data_in_leaf=100, feature_fraction=0.8, bagging_fraction=0.8, seed=7`, capped at 500 rounds with early stopping 30 — small enough that it can only apply a smooth, low-capacity correction on top of the base predictions rather than re-learning the problem.

Model Complexity is trees $\times$ leaves rather than a dense parameter count: 15 base fold-models (3 families $\times$ 5 folds) capped at 2000–2500 rounds, plus 5 meta fold-models capped at 500 rounds of 7-leaf trees; realized iteration counts after early stopping are **not recorded**. The step-18 node analysis shows what the stack buys: **equal-weight blend 0.78646 $\rightarrow$ meta-stacker 0.78680**, against base learners at 0.78594 / 0.78619 / 0.78492.

Training procedures

There is no hand-designed stage ladder. The training procedure is the search: 20 sequential nodes, each a full rewrite-execute-analyse cycle.

Search trajectory

Of the 20 steps, **17 scored and 3 were buggy**, with `buggy_exc_types` recording **three `TimeoutError` s** and nothing else. Total execution was **9587.4 s** (mean **479.4 s/node**, **max 1800.0 s** — the 30-minute per-node cap).

Step	CV R ²	Node (per AIDE's report)
0	0.78569	LightGBM baseline, 5-fold
1	0.77901	Ridge on engineered features
2	buggy	50/50 LightGBM + CatBoost blend $\rightarrow$ <code>TimeoutError</code> at 1800 s
3	0.78643	LightGBM + XGBoost 0.5/0.5 blend
4	0.78644	LightGBM + XGBoost, OOF-optimized weight
5	0.77901	Random Forest
6	0.75124	KNN, k = 25, distance-weighted
7	0.77629	PyTorch MLP, 3-layer
8	0.78647	LGB + XGB + CatBoost, linear-regression blend
9	0.78547	LGB + CatBoost, 3-fold (time-constrained)
10	0.78556	+ target encoding, 3-fold
11	0.7857	3-model blend + engineered features, 3-fold
12	0.78584	3-model + LightGBM meta-stacker, 3-fold

Step	CV R ²	Node (per AIDE's report)
13	0.7866	3-model + meta-stacker, 5-fold restored
14	buggy	+ interaction features at 5-fold → TimeoutError at 1800 s
15	0.78547	reduced-cost fallback
16	0.78611	+ interaction features (regression)
17	0.78652	+ target encoding, meta-stacker only
18	0.7868	lower learning rates, more rounds — champion
19	buggy	seed-bagging LGB and XGB → TimeoutError at 1800 s

The whole 20-step search moved the metric by:

$0.7868 - 0.78569 = 0.00111$ (first scored node → champion)
 $0.00111 / 0.78569 = 0.00141 \rightarrow 0.14\%$ relative R² gain

Two things about how that gain was obtained deserve stating.

First, the largest late improvement was a compute-utilization fix, not a modelling insight. The step-18 plan opens by observing that the previous best run "only used 361s out of the 1800s budget, meaning substantial computational headroom remains unused", and proposes a single atomic change: lower every base learner's learning rate (0.03 for LGB/XGB, 0.04 for CatBoost) and proportionally raise round budgets (2500 / 2500 / 2000) and early-stopping patience (80). That is the entire delta between step 13 and the champion:

$0.7868 - 0.7866 = 0.00020$ (step 13 → step 18)

It is a genuinely good agentic move — noticing an unspent resource and converting it into model capacity — but it is bookkeeping, not modelling, and it accounts for roughly a fifth of everything the search achieved.

Second, the search burned more than half its wall-clock on nodes that returned nothing. Three timeouts at the full cap:

$3 \times 1800.0 = 5400.0$ s consumed by timeouts
 $5400.0 / 9587.4 = 0.5632 \rightarrow 56.3\%$ of all execution time produced no metric at all

The three failures are all the same failure. Step 2 tries a 5-fold LGB + CatBoost blend at 2000 rounds each and dies "during fold 5". Step 14 tries the 5-fold three-model stack plus interaction features and dies "partway through fold 4". Step 19 tries seed-bagging two base models on top of the champion and dies "never completed fold 5". In each case the post-mortem is accurate and the prescription is correct — cut rounds, cut folds, drop a model — and in each case the next expensive design is proposed without a cost model. The agent estimates quality well and cost badly.

The run ends on a failure. Step 19, the final node, timed out, so the champion is the last node that scored rather than the search's terminal state. Combined with the step-2 timeout, AIDE lost both its first real ensembling attempt and its final refinement attempt to the same cause. The counterfactual is worth noting honestly: the seed-bagging idea at step 19 was reasonable, was proposed against the correct incumbent, and was never evaluated — the run has no evidence about whether it would have helped.

There is one further conflict worth surfacing: AIDE's own `report.md` opens by saying "Fourteen design iterations were conducted", while the journal records **20 executed nodes, 17 of them scored**. The self-report undercounts, most likely because it narrates only the nodes that returned a metric worth tabulating. The counts in this report are the journal's.

Specification fields

The **Loss Function** is squared error at both levels — LightGBM and the meta-learner use `objective="regression", metric="rmse", XGBoost reg:squarederror, CatBoost loss_function="RMSE"` — while **model selection is on R^2** (`r2_score` on the OOF vector). The two are consistent: for a fixed target, maximizing R^2 and minimizing MSE are the same optimization. The **Optimization Algorithm** is gradient boosting (histogram-based; leaf-wise for LightGBM, depth-wise for XGBoost, symmetric for CatBoost) — **not** SGD or ADAM. The **Learning Rate** is the boosting shrinkage: **0.03** for LightGBM and XGBoost, **0.04** for CatBoost, **0.05** for the meta-learner — and, as noted above, lowering it from the previous configuration is the champion's contribution. There is **no Learning Rate Scheduler** (N/A — GBDT convergence is governed by 80-round early stopping on fold RMSE, not a schedule; the "lower LR, more rounds" trade at step 18 is a manual capacity choice, not a schedule) and **no Batch Size** (N/A — full-dataset histogram boosting, not mini-batched; the MLP at step 7 was the only mini-batched candidate and it scored 0.77629).

Training Duration across the search was **9587.4 s** at **mean 479.4 s** per node against a **1800.0 s** cap, with 56.3% burned on timeouts as computed above; the champion node itself completed inside budget, and its analysis records 793 s used of 1800 s available. **Training Memory** is **not recorded**.

Transfer Learning is N/A (no pretrained weights); the AIDE analogue is **within-run journal memory** — every plan names the incumbent and its score before proposing one atomic change, which is exactly how step 18 knew both the score to beat and the compute headroom to exploit. That memory does not cross competitions. **Data Augmentation** is N/A (tabular); the analogues attempted were feature engineering (tested twice, rejected once and timed out once), fold-safe target encoding (0.78652, rejected), and seed bagging (timed out).

Reproducibility Standards are moderate. Seeds are fixed throughout — 42 for the base fold split and all three base learners, 123 for the stacking split, 7 for the meta-learner — and the pipeline is deterministic given those, so `best_solution.py` is re-runnable end to end. There is **no digit-for-digit rebuild gate** in the AIDE lane; the artifact of record is the emitted script, not a verified reconstruction. The champion configuration was chosen by comparing 17 scored candidates on the same 5-fold partition, so 0.7868 carries the usual selection optimism.

Inference procedures

Decision Threshold is N/A — a continuous regression target, so raw real-valued predictions are submitted. **Post-processing** is **none**: predictions are written straight through with no clipping to the observed [19.6, 100] range and no rounding, which is a small missed opportunity given the target is known to be bounded, though at $R^2 \approx 0.79$ the number of out-of-range predictions is presumably negligible (**not measured**). The inference path is two-stage: each base learner accumulates `predict(X_test) / 5` across its five folds, those three averaged vectors plus the raw test numerics form the stacker's input frame, and the five meta fold-models each contribute `predict(stack_test) / 5`. The final vector is written into a copy of `sample_submission.csv`, preserving id order. **Inference Duration** and **Inference Memory** are **not recorded**.

Evaluation & Benchmarking

Task for Performance Evaluation. Predict held-out students' exam scores. **Performance Metrics.** Locally, R^2 (maximize): AIDE's champion reaches **CV R^2 0.7868** at step 18, from a first scored node of 0.78569 — the 0.00111 / 0.14% gain computed above. The scored trajectory's full range is dominated by the non-GBDT probes rather than by any modelling progress:

```
0.7868 - 0.75124 = 0.03556    (champion - worst scored node, the KNN at step 6)
0.7868 - 0.78643 = 0.00037    (champion - the first two-model blend, step 3)
```

In other words, everything after step 3 — fifteen further nodes, including all three timeouts — is worth 0.00037 R^2 . The competition's signal is captured almost entirely by "blend two boosters", and the meta-stacker plus the learning-rate retune are refinements on the fourth decimal place.

On the leaderboard scale (see the source note above — this is a minimize-direction error metric, not R^2):

```
8.72191 - 8.69449 = 0.02742    (private LB - public LB)
```

Performance Benchmarking. All three agent lanes submitted to the same private split:

Agent	Approach	Private LB
Our from-scratch agent	GBDT pool + tree search	8.71733
AIDE	20-step tree search, 3 GBDTs + LightGBM meta-stacker	8.72191
NVIDIA	reproduce-agent lane	8.73239

```
8.72191 - 8.71733 = 0.00458    (our agent over AIDE)
8.73239 - 8.72191 = 0.01048    (AIDE over NVIDIA)
```

Both recorded verdicts go the same way — `local_winner = mine`, `lb_winner = mine` — so our from-scratch agent takes this episode. AIDE lands second of three, closer to our agent than NVIDIA is to AIDE:

the gap AIDE gives up (0.00458) is under half the gap it takes (0.01048). Its submission ranks **1088 / 4319, the 74.8th percentile**.

The honest verdict on this run is mixed. Architecturally it is AIDE's best work in the batch — it is the only one of the four that discovered a genuine two-level stack and demonstrated it beating both fixed and linear-optimized blends of the same components, and the step-18 "unused budget → more capacity" move is the kind of resource reasoning one wants from an agent. Operationally it is the worst: 56.3% of a two-and-a-half-hour compute allocation went to three timeouts that each restated a cost lesson the agent had already written down, and the search terminated on a failed node with an unevaluated idea on the table.

Fields marked "not recorded": realized boosting-iteration counts after early stopping; training peak memory; inference duration and memory; the count of out-of-range predictions left unclipped. Source conflicts surfaced rather than reconciled: (1) `facts_aide.json` labels the score-table metric `R2 / max` while its leaderboard columns are on a ~8.7 minimize-direction scale, so local and LB figures are reported separately; (2) AIDE's own `report.md` states "Fourteen design iterations" against the journal's 20 executed / 17 scored nodes.